

**Министерство цифрового развития, связи и массовых коммуникаций Российской
Федерации**

Ордена Трудового Красного Знамени федеральное государственное бюджетное
образовательное учреждение высшего образования
«Московский технический университет связи и информатики»

Кафедра Программная инженерия

ОТЧЁТ

по учебной практике (технологической)

Выполнил студент группы БПИ2403

Занин Никита Вячеславович

Проверила: Мосева М. С

Москва
2026

ВВЕДЕНИЕ

В период с 23.06.2026 по 06.07.2026 была пройдена учебная практика по разработке программного обеспечения. В рамках практики выполнялась роль инженера-разработчика серверной части (Backend Engineer) в проекте интеллектуальной поисковой системы по внутренней базе знаний университета. Целью практики было получение практических навыков проектирования и реализации REST API микросервисного веб-приложения на FastAPI, интеграции с поисковым движком Elasticsearch и реляционной базой данных PostgreSQL, реализации бизнес-логики обработки документов (парсинг, разбиение на чанки, индексация) и полнотекстового поиска с ранжированием результатов.

Этап 1: Инициализация приложения и реализация загрузки документов (23.06.2026 – 24.06.2026)

Период: 2 дня

***Задание:** Изучение технической документации и OpenAPI-спецификаций, инициализация FastAPI-приложения и реализация эндпоинта загрузки документов с валидацией.*

Содержание работы: Были изучены цели и задачи проекта, микросервисная архитектура приложения и распределение обязанностей в команде. Инициализировано приложение на фреймворке FastAPI: настроена точка входа main.py, конфигурация через pydantic-settings и подключение к PostgreSQL средствами асинхронного SQLAlchemy. Реализован REST API эндпоинт POST /api/v1/documents/upload для загрузки файлов. Разработан модуль валидации (file_validation): проверка формата (PDF, DOCX) и размера (не более 20 МБ) с возвратом HTTP 400 и описанием ошибки при несоответствии. Для каждого загружаемого документа генерируется уникальный идентификатор UUID, метаданные сохраняются в БД.

Этап 2: Извлечение текста из документов и разбиение на чанки (25.06.2026 – 26.06.2026)

Период: 2 дня

***Задание:** Реализация модулей извлечения текста из PDF- и DOCX-документов и сегментации извлечённого текста.*

Содержание работы: Разработан сервис извлечения текста (document_parser) с использованием библиотеки pdfplumber для PDF-документов и python-docx для DOCX. Обеспечено сохранение номеров страниц для корректной навигации по результатам поиска. Реализован модуль сегментации (chunker): извлечённый текст разбивается на чанки по ~500 символов с сохранением границ предложений, что повышает полноту поиска и сохраняет контекст. Проведена обработка краевых случаев — пустые файлы, документы с нарушенным форматированием и нестандартными кодировками, для которых предусмотрен корректный перехват исключений.

Этап 3: Интеграция с Elasticsearch и индексация документов (27.06.2026)

Период: 1 день

***Задание:** Настройка Elasticsearch с русскоязычным анализатором и реализация функции индексации чанков документов.*

Содержание работы: Настроено подключение к Elasticsearch через асинхронный клиент. Создан индекс documents с русскоязычным анализатором (analysis-ru) и маппингом полей для полнотекстового поиска. Реализована функция индексации (elasticsearch_service): каждый чанк документа сохраняется в Elasticsearch вместе с метаданными — file_name, page_number, chunk_id и text. Настроена связка процесса загрузки, парсинга, сегментации и индексации в единый конвейер обработки документа (document_service), запускаемый после успешной загрузки файла.

Этап 4: Реализация эндпоинта полнотекстового поиска (28.06.2026 – 29.06.2026)

Период: 2 дня

***Задание:** Разработка эндпоинта полнотекстового поиска и формирование структуры ответа с ранжированием результатов.*

Содержание работы: Реализован эндпоинт GET /api/v1/search?q={query}, выполняющий запрос к Elasticsearch с использованием multi_match по полю text с настройкой fuzziness: AUTO. Результаты поиска возвращаются в формате JSON с полями chunk_id, file_name, page, text и score (оценка релевантности), а совпавшие фрагменты сопровождаются подсветкой (highlight) для отображения на фронтенде. Реализован эндпоинт возврата списка загруженных документов с их названиями, датами загрузки и статусами. Валидация входных и выходных данных обеспечена через Pydantic-схемы, что гарантирует стабильный контракт API для фронтенд-части.

Этап 5: Кеширование запросов и сохранение истории поиска (30.06.2026 – 01.07.2026)

Период: 2 дня

***Задание:** Реализация кеширования частых поисковых запросов и сохранения истории поиска, оптимизация запросов.*

Содержание работы: Реализован модуль кеширования (cache) на базе Redis: результаты поисковых запросов сохраняются в кеше с временем жизни (TTL) 5 минут, и при повторном поступлении того же запроса ответ отдаётся из кеша, что снижает нагрузку на Elasticsearch и уменьшает время отклика. Реализовано

сохранение истории поисковых запросов в PostgreSQL (модель search_history). Проведена оптимизация запросов к Elasticsearch и профилирование бизнес-логики для обеспечения стабильной производительности под нагрузкой.

Этап 6: Документирование API и обработка ошибок (02.07.2026 – 03.07.2026)

Период: 2 дня

***Задание:** Документирование эндпоинтов через OpenAPI и реализация единообразной обработки ошибок с корректными HTTP-статусами.*

Содержание работы: Все эндпоинты задокументированы через OpenAPI 3.0; интерактивная документация Swagger UI доступна по адресу /docs. Реализована централизованная обработка исключений (exceptions), обеспечивающая возврат корректных HTTP-статусов: 200 OK при успешном выполнении, 400 Bad Request при ошибке валидации, 404 Not Found при отсутствии ресурса и 500 Internal Server Error при внутренней ошибке. Настроено структурированное логирование (logging_config). Код приведён в соответствие со стандартом PEP 8, публичные функции и классы снабжены описаниями параметров и возвращаемых значений.

Этап 7: Экспорт метрик, тестирование и финальная отладка (04.07.2026 – 06.07.2026)

Период: 2 дня

Задание: *Подключение экспорта метрик для мониторинга, итоговая отладка серверной части, написание тестов и подготовка отчётной документации.*

Содержание работы: Подключён экспорт метрик приложения через prometheus-fastapi-instrumentator: эндпоинт /metrics предоставляет Prometheus данные о количестве запросов к /search и времени ответа. Реализован эндпоинт проверки работоспособности /health, используемый оркестратором и скриптом инициализации. Выполнена итоговая отладка всех серверных модулей, проверена корректность обработки полного жизненного цикла документа — загрузка, парсинг, индексация, поиск. Реализовано тестирование на трёх уровнях: юнит-тесты (pytest) с покрытием более 50%, E2E-тесты (Playwright) — сквозной сценарий от загрузки до поиска, нагрузочные тесты (locust) с 50 виртуальными пользователями. Метрика качества поиска — Precision@3 = 100% на 10 эталонных запросах. Сформирована финальная отчётность по серверной части проекта и составлен список приобретённых ИТ-компетенций.

ЗАКЛЮЧЕНИЕ

В ходе учебной практики были успешно выполнены все запланированные этапы работы. В рамках роли Backend Engineer спроектирована и реализована серверная часть микросервисной поисковой системы на FastAPI. Реализованы загрузка и валидация документов, извлечение текста из PDF и DOCX, сегментация и индексация в Elasticsearch с русскоязычным анализатором, полнотекстовый поиск с ранжированием и подсветкой результатов, кеширование через Redis и сохранение истории запросов в PostgreSQL. Все эндпоинты задокументированы через OpenAPI, обеспечена корректная обработка ошибок и экспорт метрик. Серверная часть покрыта тестами, задокументирована и полностью готова к эксплуатации.

Приобретённые навыки: разработка REST API на FastAPI, проектирование бизнес-логики и сервисного слоя, работа с реляционной БД PostgreSQL через асинхронный SQLAlchemy, интеграция с Elasticsearch (маппинг, анализаторы, multi_match, highlight), извлечение текста из PDF/DOCX (pdfplumber, python-docx), кеширование через Redis, валидация данных через Pydantic, документирование API (OpenAPI/Swagger), обработка исключений и логирование, соблюдение стандарта PEP 8, работа с Git, написание юнит-, E2E- и нагрузочных тестов (pytest, Playwright, locust), контейнеризация через Docker Compose, настройка CI/CD в GitHub Actions.

Статистика: Общая продолжительность практики — 12 дней, количество этапов — 7, реализовано API-эндпоинтов: 4 (загрузка, поиск, список документов, health), разработано сервисных модулей: 7 (валидация, парсинг, сегментация, индексация, поиск, кеш, оркестрация обработки), поддерживаемых форматов документов: 2 (PDF, DOCX), интеграций с внешними системами: 3 (PostgreSQL, Elasticsearch, Redis). **Ссылка на репозиторий:** <https://github.com/terminatornikita/university-search-mtusi>